

AquaGuardian AI — Proof-of-Concept Test Report

GPIW 2026 Submission · Supporting Evidence · Track A (TRL 1–3) · Digitalisation, Automation and Smart Water Management

1. Purpose and Scope

This report documents the automated test suite and comparative simulation results for the AquaGuardian AI proof of concept, generated directly from the code in the submitted repository. It is **not** a third-party certification, an independent audit, or a field test report — no external test laboratory, standards body, or customer was involved. Every figure below was produced by running the project's own automated tests (pytest) and simulation scripts, and is reproducible by any reviewer with the repository and a Python 3.11+ environment.

This report is offered as the strongest evidence currently available at TRL 3: not proof the system works on real infrastructure, but proof the claims made elsewhere in this application are backed by running code rather than assertion.

2. Test Environment

Parameter	Value
Python version	3.12.3
Test framework	pytest 9.1.1
Operating system	Linux x86_64
Execution mode	Fully offline, deterministic (fixed random seeds throughout)
Reproducibility	Dockerfile and docker-compose.yml included in repository root
Full suite runtime	≈ 193 seconds

3. Automated Test Suite Composition

135 tests were collected across eight test modules, spanning both the AquaGuardian application layer and the bundled hexa_tpu_rt_sim hardware/network/middleware simulator.

Module	Tests	What it covers
tests/test_engine.py	3	Core decision engine: risk detection per scenario, validated-action selection, rejection of 'do nothing' under critical cases
tests/test_comparison.py	3	Three-strategy comparative aggregation (reactive / detection-only / closed-loop)
tests/test_api_integration.py	7	FastAPI endpoints, console scenario wiring, evidence metrics are backend-derived (not hard-coded)
tests/test_hardware_bridge.py	5	Live reflex-kernel, network, and MEC-layer benchmark wiring; determinism for a fixed seed

Module	Tests	What it covers
hexa_tpu_rt_sim/tests/test_core.py	68	Core simulator primitives (sensor models, scheduling, DMA/AXI-style memory paths)
hexa_tpu_rt_sim/tests/test_reflex_kernel.py	17	Reflex-kernel deadline compliance across domains
hexa_tpu_rt_sim/tests/test_middleware_mec.py	23	oneM2M resource-tree CRUD, MEC platform registration and dispatch (3 skipped — see below)
hexa_tpu_rt_sim/tests/test_water_cognitive_engine.py	9	Water-domain MEC cognitive engine (Bat Forecaster, Squid Controller, Hermit-Crab veto)

Result: 132 passed, 3 skipped, 0 failed (10 subtests passed within the 132). The 3 skipped tests are in `test_middleware_mec.py` and require a live external oneM2M CSE server reachable over HTTP (e.g. via `acmecse`) — not available in this execution environment. They are not silently ignored: `pytest` reports the exact skip reason and the command needed to enable them, and they test the HTTP binding layer specifically, not the resource-tree logic itself (which is covered by the passing tests and exercised live in Section 5).

4. Comparative Strategy Evaluation

Three decision strategies were run across the same 5 simulated water-risk scenarios (pipeline leak, pump degradation, water contamination, drought stress, wildfire risk), each under 4 stress profiles (nominal through extreme, including sensor noise and communication loss) — 20 runs per strategy, 60 total. Source: `aquaguardian/comparison.py`, reproducible via `pytest tests/test_comparison.py`.

Strategy	Avg. water loss (L)	Avg. stress-test pass rate	Avg. safety risk score	Avg. service disruption (min)	Avg. response delay (s)
Reactive baseline	420.96	0%	0.100	20.2	300.0
Detection-only AI	194.33	50%	0.134	3.8	2.0
AquaGuardian closed-loop	117.04	100%	0.112	10.0	0.25

Two ways to read the water-loss improvement, both reported for transparency: comparing the *aggregate averages* above, AquaGuardian reduces water loss by 72.2% versus the reactive baseline (117.04 vs 420.96 L). Averaging the *per-scenario percentage reductions* instead (72.9%, 99.9%, 68.4%, 79.1%, 51.9% across the five scenarios) gives 74.4% — the figure used in the accompanying pitch deck. Both are legitimate, differently-weighted summaries of the same underlying data; neither is cherry-picked.

4.1 Per-Scenario Breakdown (vs. reactive baseline)

Scenario	Water loss reduction	Response time reduction	Stress pass-rate gain	Safety risk change
Pipeline leak	72.9%	99.9%	+100 pts	0%
Pump degradation	99.9%	99.9%	+100 pts	−111.1%*
Water contamination	68.4%	99.9%	+100 pts	0%
Drought stress	79.1%	99.9%	+100 pts	0%

Scenario	Water loss reduction	Response time reduction	Stress pass-rate gain	Safety risk change
Wildfire risk	51.9%	99.9%	+100 pts	0%

* Disclosed, not omitted: in the pump-degradation scenario, AquaGuardian's selected plan carries a **higher** nominal safety-risk score than the reactive baseline under this scoring model, even though it passes stress testing where the reactive plan fails. This likely reflects the closed loop choosing a more decisive action that trades a nominal risk-score cost for actually passing validation under adversarial conditions — but that interpretation is not yet independently confirmed, and the result is reported as-is rather than adjusted.

4.2 Stress-Test Validator Coverage

Separately from the strategy comparison above, the stress-test validator itself was run against 80 (scenario, stress-profile, candidate-action) combinations, covering 12 distinct candidate actions across the 5 scenarios. Source: `outputs/stress_matrix.json`.

Total runs	Passed	Failed	Pass rate
80	42	38	52.5%

This is a distinct metric from the 0%→100% pass rate in Section 4: that number describes whether each strategy's *selected* plan passes; this one describes the validator's overall discriminating power across *every* candidate action tried, including ones no strategy would actually select. A 52.5% pass rate here indicates the stress-test policy is not a rubber stamp — it rejects roughly half of all candidate actions evaluated, nominal ones included.

5. Hardware and Protocol-Layer Validation

The application's Green Box / edge-AI claims are backed by `hexa_tpu_rt_sim`, a separately built architecture simulator bundled in the repository. `app/hardware_bridge.py` calls it live — the numbers below are computed fresh on each run, not cached constants, and are re-verified by `tests/test_hardware_bridge.py`.

Layer	What was tested	Result
Reflex kernel	Water-hammer / pipe-burst trigger, 200 samples, 1ms hard deadline	Max latency 0.00003ms (30ns); 0 deadline misses
Edge → MEC network	Modeled as 5G NR URLLC, 200 transmissions, 10ms SLA target	p99 latency 4.23ms; SLA met on all samples
MEC cognitive layer	Worsening leak (severity 0.8) — Bat Forecaster + Squid Controller + Hermit-Crab veto	Selected: staged_valve_shutdown; vetoed: emergency_full_shutdown
oneM2M middleware	AE registration, 2 container creates, 1 event CREATE, 1 explicit RETRIEVE (TS-0004 primitives)	All primitives returned success (2001/2000); 5 requests logged

Scope note, from `hexa_tpu_rt_sim`'s own documentation: these are cycle-accurate deadline checks and a statistical network model running in software — not measurements from real embedded hardware or a real radio, and the oneM2M layer is an in-memory resource tree, not a live wire-protocol exchange with an external CSE (see Section 3's note on the 3 skipped HTTP-binding tests). The `ReflexKernel`, `BatForecaster`, and `SquidController` classes exercised here are reused unmodified from the project's separate aviation domain — evidence the architecture generalizes rather than being a water-specific script.

6. Known Limitations and Disclosures

- All results are generated by a deterministic software simulator. No sensor, actuator, or hydraulic network used in this testing is physical.
- No independent or third-party test laboratory, standards body, or certification authority was involved in producing any result in this report.
- One comparative metric (safety-risk score, pump-degradation scenario) does not favor AquaGuardian and is reported as-is in Section 4.1, not adjusted or omitted.
- 3 of 135 tests are skipped in this environment because they require a live external oneM2M CSE server; they test an HTTP binding layer, not the resource-tree logic exercised live in Section 5.
- No field trial, pilot deployment, or real-infrastructure data exists yet for this project. This report supports a TRL 3 classification, not a higher one.

7. Reproducing These Results

From the repository root, with dependencies installed (`pip install -r requirements.txt -r requirements-dev.txt`):

```
pytest -q – runs the full 135-test suite
python3 -c "from app import hardware_bridge; print(hardware_bridge.run_water_hardware_benchmark())"
– regenerates Section 5
python3 -m aquaguardian.comparison or pytest tests/test_comparison.py – regenerates Section 4
```

AquaGuardian AI — GPIW 2026 Submission · Islam Abdelmogood, Solo Founder · Report generated from repository state; all figures reproducible.